

57. Fetch: Abort

`fetch` promise return karta hai, aur JS me promise ko "abort" karne ka concept nahi hai. Chalu `fetch` ko cancel karna ho (jaise user ne kuch kiya aur ab request ki zarurat nahi) to `AbortController` hai. Ye sirf `fetch` nahi, dusre async tasks ko bhi abort kar sakta hai.

`AbortController`

```
js
let controller = new AbortController();
```

Bahut simple object:

- Ek method: `abort()`
- Ek property: `signal` (jispar event listeners lagte hain)

`abort()` call hone par:

- `controller.signal` par `"abort"` event aata hai
- `controller.signal.aborted` `true` ho jaata hai

Do parties hoti hain:

1. Jo cancel hone layak kaam kar raha hai: `controller.signal` par listener lagata hai.
2. Jo cancel karta hai: zarurat par `controller.abort()` call karta hai.

```
js
let controller = new AbortController();
let signal = controller.signal;

signal.addEventListener('abort', () => alert("abort!"));

controller.abort();           // abort!
alert(signal.aborted);        // true
```

`fetch` ke saath

`signal` ko fetch ke option me pass karo:

```
js
```

```
let controller = new AbortController();
fetch(url, { signal: controller.signal });

controller.abort(); // request abort
```

Abort hone par fetch ka promise `AbortError` ke saath **reject** hota hai, to `try..catch` me handle karo:

```
js

let controller = new AbortController();
setTimeout(() => controller.abort(), 1000); // 1 second baad abort

try {
  let response = await fetch('/hang', { signal: controller.signal });
} catch (err) {
  if (err.name === 'AbortError') {
    alert("Aborted!");
  } else {
    throw err;
  }
}
```

`AbortController` scalable hai

Ek controller se **kai fetches ek saath** cancel kar sakte ho:

```
js

let controller = new AbortController();

let fetchJobs = urls.map(url => fetch(url, { signal: controller.signal }));
let results = await Promise.all(fetchJobs);

// kahin se bhi controller.abort() to sab fetches abort
```

Apne khud ke async tasks ko bhi saath me rok sakte ho, bas unme `abort` event sunna hoga:

```
js
```

```
let ourJob = new Promise((resolve, reject) => {
  // ...
  controller.signal.addEventListener('abort', reject);
});

let results = await Promise.all([...fetchJobs, ourJob]);
```

Summary

- `AbortController`: `abort()` call par `signal` par `abort` event (aur `signal.aborted = true`)).
- `fetch` isse integrated hai: `signal` option me do, `abort()` par request cancel.
- Apne code me bhi use kar sakte ho ("abort() call karo" -> "abort event suno"), bina `fetch` ke bhi.